

System Design Basics

Lesson 5: Data Storage Choices

Data Storage Choices – Study Notes

Key Concepts

1. **Relational (SQL) databases** – structured tables, fixed schemas, ACID transactions.
2. **NoSQL databases** – flexible data models (document, key value, column family, graph), eventual consistency options.
3. **Schema design & evolution** – when a rigid schema helps vs. when schema flexibility is needed.
4. **Query patterns & performance** – joins, indexing, read heavy vs write heavy workloads.
5. **Operational considerations** – scaling, maintenance, hosting options, and cost for a blogging platform.

Summary

Relational databases (e.g., PostgreSQL, MySQL) excel when data relationships are well defined, consistency is critical, and complex queries (joins, aggregations) are frequent. They enforce a fixed schema, which makes data validation straightforward and provides strong ACID guarantees—useful for core blog entities such as users, posts, comments, and permissions where referential integrity matters.

NoSQL databases (e.g., MongoDB, Cassandra, Redis) trade some of that strict consistency for schema flexibility, horizontal scalability, and high throughput read/write patterns. They are ideal for storing loosely structured or rapidly evolving data, such as article drafts, tag clouds, activity logs, or full text search indexes. Because they often avoid costly joins, they can serve high traffic read heavy scenarios (e.g., serving millions of blog views) with lower latency.

Choosing between them for a blogging platform depends on the specific subsystem: use a relational DB for the core transactional data (users, posts, comments, permissions) and complement it with a NoSQL store for ancillary, high volume, or schema fluid data (draft versions, caching, analytics, full text search). A hybrid approach lets you leverage the strengths of each technology while keeping the overall architecture simple.

Important Points

- **ACID vs. BASE**
 - Relational: **Atomicity, Consistency, Isolation, Durability** – strong guarantees.
 - NoSQL: **Basically Available, Soft state, Eventual consistency** – trade offs for scalability.
 - **Schema rigidity**
 - Relational tables require predefined columns; altering schema can be costly.
 - Document stores (MongoDB) allow each record to have its own fields, easing feature rollout.
 - **Query capabilities**
 - SQL supports complex joins, sub queries, window functions—great for reporting.

- NoSQL often provides limited ad hoc querying; you design queries up front (e.g., using indexes or materialized views).

- **Scaling**

- Relational: vertical scaling (bigger server) or read replicas; sharding is possible but complex.

- NoSQL: built in horizontal sharding/partitioning; easier to add nodes for traffic spikes.

- **Use case mapping for a blog**

| Subsystem | Recommended Store | Why? |

|-----|-----|-----|

| Users, Posts, Comments, Permissions | Relational (PostgreSQL/MySQL) | Strong integrity, foreign key constraints, transactional updates |

| Drafts & revisions | Document NoSQL (MongoDB) | Varying fields per draft, easy versioning |

| Tag cloud / search index | Document/Key Value (Elasticsearch, Redis) | Full text search, fast look ups |

| Site analytics / view counters | Column family (Cassandra) or Key Value (Redis) | Write heavy, high throughput, eventual consistency acceptable |

| Caching layer | In memory NoSQL (Redis, Memcached) | Low latency for popular posts |

- **Data consistency strategy**

- Keep the authoritative source in the relational DB.

- Use asynchronous pipelines (e.g., change data capture, message queues) to propagate updates to NoSQL stores.

- **Operational overhead**

- One database = simpler ops, but may hit limits.

- Multiple databases = more components to monitor, but each can be tuned for its workload.

Examples

Example 1 – Storing Core Blog Data in PostgreSQL

```
```sql
```

```
CREATE TABLE users (
 id SERIAL PRIMARY KEY,
 username VARCHAR(50) UNIQUE NOT NULL,
 email VARCHAR(255) UNIQUE NOT NULL,
 password_hash TEXT NOT NULL,
 created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

```
CREATE TABLE posts (
 id SERIAL PRIMARY KEY,
 author_id INT REFERENCES users(id) ON DELETE CASCADE,
 title VARCHAR(200) NOT NULL,
 body TEXT NOT NULL,
 published_at TIMESTAMP,
 created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

```
CREATE TABLE comments (
 id SERIAL PRIMARY KEY,
 post_id INT REFERENCES posts(id) ON DELETE CASCADE,
 user_id INT REFERENCES users(id) ON DELETE CASCADE,
 comment TEXT NOT NULL,
 created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

```

 id SERIAL PRIMARY KEY,
 post_id INT REFERENCES posts(id) ON DELETE CASCADE,
 author_id INT REFERENCES users(id),
 content TEXT NOT NULL,
 created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

```

**\*Explanation:** The foreign key constraints guarantee that a comment cannot exist without a valid post and user. Transactions ensure that creating a post and its initial tags (if stored in a separate table) succeed or roll back together.

## Example 2 – Drafts in MongoDB (Document Store)

```

```js
// A draft document
{
  _id: ObjectId("64a7c2f1e4b0c8d5f9a1b2c3"),
  authorId: 42,
  postId: null,           // null until published
  title: "My First Draft",
  body: "Lorem ipsum ...",
  metadata: {
    wordCount: 523,
    images: [{url:"/img/1.png", caption:"Intro"}],
    customFields: {seoTitle:"...", seoDesc:"..."}
  },
  versions: [
    { version: 1, editedAt: ISODate("2024-04-10T12:00:00Z") },
    { version: 2, editedAt: ISODate("2024-04-12T08:15:00Z") }
  ],
  createdAt: ISODate("2024-04-10T11:58:00Z")
}
---
```

***Explanation:** Each draft can have a different set of `customFields` without altering a global schema. Adding a new field for SEO metadata only requires updating the application code, not the database schema.

Quick Review

1. ****When is a relational database the better choice for a blogging platform?****
2. ****Name two scenarios where a NoSQL store adds value to the same platform.****
3. ****What does ACID stand for, and why is it important for core blog data?****
4. ****How can you keep data consistent between a relational DB and a NoSQL cache?****
5. ****What is a common trade off when opting for a NoSQL solution?****

Further Reading

- **SQL vs. NoSQL: Deep Dive** – comparative performance benchmarks.
- **CAP Theorem** – understanding consistency, availability, and partition tolerance.
- **Eventual Consistency Patterns** – read repair, write behind, and conflict resolution.
- **Hybrid Architecture Case Studies** – real world blogs and content management systems.
- **Scaling PostgreSQL** – partitioning, logical replication, and connection pooling.

